

```

import os
import json
import google.generativeai as genai
import PyPDF2
import io
import asyncio

from telegram import Update, Poll
from telegram.ext import (
    ApplicationBuilder, CommandHandler, MessageHandler,
    filters, ContextTypes, PollAnswerHandler
)

# =====
# SOZLAMALAR
# =====
TELEGRAM_TOKEN = os.environ.get("TELEGRAM_TOKEN", "BU_YERGA_TELEGRAM_TOKEN")
GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY", "BU_YERGA_GEMINI_API_KEY")

genai.configure(api_key=GEMINI_API_KEY)
model = genai.GenerativeModel("gemini-1.5-flash")

user_histories = {}
user_quizzes = {}
poll_data = {}

SYSTEM_PROMPT = """Sen o'zbek tilida ishlaydigan aqlli shaxsiy yordamchisan.
Foydalanuvchiga har qanday savolda yordam ber: tarjima, matn yozish, hisob-kitob,
Doimo o'zbek tilida javob ber. Qisqa, aniq va foydali bo'l."""

# =====
# PDF DAN MATN OLISH
# =====
def extract_text_from_pdf(pdf_bytes: bytes) -> str:
    reader = PyPDF2.PdfReader(io.BytesIO(pdf_bytes))
    text = ""
    for page in reader.pages:
        text += page.extract_text() + "\n"
    return text[:8000]

# =====
# GEMINI ORQALI JAVOB OLISH
# =====
def ask_gemini(messages: list) -> str:

```

```

# Tarixni Gemini formatiga o'tkazish
history = []
for msg in messages[:-1]:
    role = "user" if msg["role"] == "user" else "model"
    history.append({"role": role, "parts": [msg["content"]]})

chat = model.start_chat(history=history)
last_msg = SYSTEM_PROMPT + "\n\n" + messages[-1]["content"] if len(messages)
response = chat.send_message(last_msg)
return response.text

# =====
# GEMINI ORQALI SAVOLLAR TUZISH
# =====
def generate_quiz(text: str, num_questions: int = 5) -> list:
    prompt = f"""Quyidagi matndan {num_questions} ta test savoli tuz.

QOIDALAR:
- Har bir savolda 4 ta javob varianti bo'lsin
- Faqat 1 ta to'g'ri javob
- O'zbek tilida yoz
- FAQAT JSON qaytap, boshqa hech narsa yozma

FORMAT:
[
    {{
        "question": "Savol?",
        "options": ["A variant", "B variant", "C variant", "D variant"],
        "correct": 0,
        "explanation": "Izoh"
    }}
]

MATN:
{text}"""

    response = model.generate_content(prompt)
    raw = response.text.strip()

    if "```json" in raw:
        raw = raw.split("```json")[1].split("```")[0].strip()
    elif "```" in raw:
        raw = raw.split("```")[1].split("```")[0].strip()

    return json.loads(raw)

# =====

```

```

# /start
# =====
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_id = update.effective_user.id
    user_histories[user_id] = []
    await update.message.reply_text(
        "👋 Salom! Men sizning shaxsiy AI yordamchingizman!\n\n"
        "💬 Oddiy suhbat: Har qanday savol yozing\n"
        "📄 PDF Test: PDF fayl yuboring → avtomatik test!\n\n"
        "📌 Buyruqlar:\n"
        "/start - Qayta boshlash\n"
        "/clear - Suhbatni tozalash\n"
        "/help - Yordam\n\n"
        "Boshlaylik! 🚀"
    )

# =====
# /help
# =====
async def help_command(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "💡 Nima qila olaman:\n\n"
        "🌐 Tarjima - 'inglizchaga tarjima qil: ...'\n"
        "✍️ Matn yozish - '... haqida matn yoz'\n"
        "🧮 Hisoblash - 'quyidagini hisobla: ...'\n"
        "👤 Maslahat - 'maslahat ber: ...'\n\n"
        "📄 PDF Test:\n"
        "1. PDF fayl yuboring\n"
        "2. Nechta savol kerakligini yozing (1-10)\n"
        "3. Test boshlanadi!\n"
        "4. Oxirida natija chiqadi ✅"
    )

# =====
# /clear
# =====
async def clear(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_id = update.effective_user.id
    user_histories[user_id] = []
    await update.message.reply_text("✅ Suhbat tarixi tozalandi!")

# =====
# PDF QABUL QILISH
# =====
async def handle_document(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_id = update.effective_user.id
    document = update.message.document

```

```

if not document.file_name.lower().endswith(".pdf"):
    await update.message.reply_text("⚠️ Faqat PDF fayl yuboring!")
    return

await update.message.reply_text("📄 PDF qabul qilindi! O'qilyapti...")

try:
    file = await context.bot.get_file(document.file_id)
    pdf_bytes = await file.download_as_bytearray()
    text = extract_text_from_pdf(bytes(pdf_bytes))

    if len(text.strip()) < 100:
        await update.message.reply_text("⚠️ PDF dan matn o'qib bo'lmadi.")
        return

    context.user_data["pdf_text"] = text
    context.user_data["waiting_for_count"] = True

    await update.message.reply_text(
        f"✅ PDF muvaffaqiyatli o'qildi!\n"
        f"📄 Matn hajmi: {len(text)} belgi\n\n"
        f"? Nechta savol tuzishni istaysiz? (1 dan 10 gacha)"
    )

except Exception as e:
    await update.message.reply_text(f"❌ Xatolik: {str(e)}")

# =====
# TEST SAVOLINI YUBORISH
# =====
async def send_quiz_question(context, user_id, chat_id):
    quiz = user_quizzes.get(user_id)
    if not quiz:
        return

    idx = quiz["current"]
    total = len(quiz["questions"])

    if idx >= total:
        correct = quiz["correct"]
        wrong = total - correct
        percent = int((correct / total) * 100)
        emoji = "🏆" if percent >= 80 else "👍" if percent >= 60 else "📚"

    result = (
        f"{emoji} Test yakunlandi!\n\n"

```

```

        f"📊 Natija: {correct}/{total}\n"
        f"✅ To'g'ri: {correct}\n"
        f"❌ Noto'g'ri: {wrong}\n"
        f"📈 Foiz: {percent}%\n\n"
    )
    if percent >= 80:
        result += "🎉 Ajoyib! Bilimlaringiz zo'r!"
    elif percent >= 60:
        result += "👏 Yaxshi! Biroz takrorlang."
    else:
        result += "📖 Mavzuni qayta o'rganing. Baribir zo'rsiz!"

    del user_quizzes[user_id]
    await context.bot.send_message(chat_id=chat_id, text=result)
    return

question = quiz["questions"][idx]
msg = await context.bot.send_poll(
    chat_id=chat_id,
    question=f"❓ {idx+1}/{total}. {question['question']}",
    options=question["options"],
    type=Poll.QUIZ,
    correct_option_id=question["correct"],
    explanation=question.get("explanation", ""),
    is_anonymous=False,
    open_period=30
)

poll_data[msg.poll.id] = {
    "user_id": user_id,
    "chat_id": chat_id,
    "correct_option": question["correct"]
}

# =====
# POLL JAVOBINI QABUL QILISH
# =====
async def handle_poll_answer(update: Update, context: ContextTypes.DEFAULT_TYPE):
    answer = update.poll_answer
    poll_id = answer.poll_id
    user_id = answer.user.id

    if poll_id not in poll_data:
        return

    data = poll_data[poll_id]
    quiz = user_quizzes.get(user_id)

```

```

if not quiz:
    return

if answer.option_ids and answer.option_ids[0] == data["correct_option"]:
    quiz["correct"] += 1

quiz["current"] += 1
del poll_data[poll_id]

await asyncio.sleep(1)
await send_quiz_question(context, user_id, data["chat_id"])

# =====
# MATN XABARLAR
# =====
async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_id = update.effective_user.id
    chat_id = update.effective_chat.id
    text = update.message.text.strip()

    if context.user_data.get("waiting_for_count"):
        try:
            count = int(text)
            if count < 1 or count > 10:
                await update.message.reply_text("⚠️ 1 dan 10 gacha son kiriting!")
                return

            context.user_data["waiting_for_count"] = False
            pdf_text = context.user_data.get("pdf_text", "")

            await update.message.reply_text(f"⌚ {count} ta savol tuzilmoqda... ⚡
            await context.bot.send_chat_action(chat_id=chat_id, action="typing")

            questions = generate_quiz(pdf_text, count)

            user_quizzes[user_id] = {
                "questions": questions,
                "current": 0,
                "correct": 0
            }

            await update.message.reply_text(
                f"✅ {len(questions)} ta savol tayyor!\n"
                f"🎯 Test boshlanmoqda...\n"
                f"🕒 Har bir savolga 30 soniya!"
            )

```

```

        await asyncio.sleep(1)
        await send_quiz_question(context, user_id, chat_id)
        return

    except ValueError:
        await update.message.reply_text("⚠ Faqat son kiriting (1-10)!")
        return

# Oddiy suhbat
if user_id not in user_histories:
    user_histories[user_id] = []

await context.bot.send_chat_action(chat_id=chat_id, action="typing")
user_histories[user_id].append({"role": "user", "content": text})

if len(user_histories[user_id]) > 20:
    user_histories[user_id] = user_histories[user_id][-20:]

try:
    reply = ask_gemini(user_histories[user_id])
    user_histories[user_id].append({"role": "assistant", "content": reply})
    await update.message.reply_text(reply)

except Exception as e:
    await update.message.reply_text(f"⚠ Xatolik: {str(e)}")

# =====
# BOTNI ISHGA TUSHIRISH
# =====
def main():
    print("🤖 Bot ishga tushmoqda...")
    app = ApplicationBuilder().token(TELEGRAM_TOKEN).build()

    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("help", help_command))
    app.add_handler(CommandHandler("clear", clear))
    app.add_handler(MessageHandler(filters.Document.PDF, handle_document))
    app.add_handler(PollAnswerHandler(handle_poll_answer))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, handle_message))

    print("✅ Bot ishga tushdi!")
    app.run_polling()

if __name__ == "__main__":
    main()

```

